
Préparation au Concours National Commun — Informatique MP

Corrigé détaillé

Filière MP — Durée : 4 heures

Voyage d'un paquet : de l'adresse IP au cadenas du navigateur

Les protocoles TCP/IP et HTTP/HTTPS

Auteur : **Pr Agrégé El Hadiq Zouhair**

cpgeacademy.org — Tous droits réservés © cpgeacademy.org

Avis important. Ce document est une **initiative personnelle** du professeur, à but strictement pédagogique. Il ne s'agit **pas** d'un sujet officiel du Concours National Commun.

Note de vérification. L'intégralité du code Python de ce corrigé a été exécutée ; les fonctions ont été confrontées à des implantations naïves de référence sur plusieurs milliers de jeux d'essai aléatoires (arbre de préfixes, fusion d'intervalles, cache LRU, programmation dynamique), et les requêtes SQL ont été exécutées sur une base réelle. Les exemples numériques des questions 16 et 17 sont ceux des [RFC 1624](#) et [RFC 1071](#).

Pour aller plus loin. [cpgeacademy.org](#).

Partie I — Adressage IPv4 et agrégation CIDR

Question 1. Il s'agit d'une écriture en base 256 sur exactement quatre chiffres.

```
1 def ip_vers_entier(s):
2     x = 0
3     for c in s.split('.'):
4         x = x * 256 + int(c)
5     return x
6
7 def entier_vers_ip(x):
8     o = []
9     for _ in range(4):
10        o.append(str(x % 256))
11        x = x // 256
12    return '.'.join(reversed(o))
```

Les deux boucles font exactement quatre tours : la complexité est $\Theta(1)$. (`ip_vers_entier` applique le schéma de Horner en base 256 ; `entier_vers_ip` extrait les chiffres du poids faible vers le poids fort, d'où le `reversed`.)

Question 2.

```
1 def masque(n):
2     if n == 0:
3         return 0
4     return ((1 << n) - 1) << (32 - n)
```

Démonstration. Pour $n \geq 1$, $2^n - 1$ est l'entier dont les n bits de poids faible valent 1. Le décaler de $32 - n$ positions vers la gauche revient à le multiplier par 2^{32-n} : on obtient $(2^n - 1)2^{32-n} = 2^{32} - 2^{32-n}$, entier dont les n bits de poids fort valent 1 et les $32 - n$ autres 0. Le cas $n = 0$ est

traité à part car $1 \ll 0 - 1 = 0$ conviendrait, mais un décalage de 32 n'est pas défini sur un mot de 32 bits dans la plupart des langages ; on garde donc le test pour la portabilité. $\square$

Question 3.

```

1 def adresse_reseau(a, n):
2     return a & masque(n)
3
4 def diffusion(a, n):
5     return a | (0xFFFFFFFF ^ masque(n))

```

Démonstration. Notons $r = \text{adresse_reseau}(a, n)$. Par définition du et bit à bit, r a les mêmes n bits de poids fort que a et ses $32 - n$ bits de poids faible nuls. De même, $\text{diffusion}(a, n)$ a les mêmes n bits de poids fort que a et ses $32 - n$ bits de poids faible tous égaux à 1, c'est-à-dire vaut $2^{32-n} - 1$ sur cette partie. Comme r vaut 0 sur ces mêmes bits et que les deux nombres coïncident ailleurs,

$$\text{diffusion}(a, n) - r = 2^{32-n} - 1.$$

Ce sont bien le minimum et le maximum de $\mathcal{A}(p/n)$: écrire $a = r + u$ avec $0 \leq u \leq 2^{32-n} - 1$ établit que $\mathcal{A}(p/n) = \llbracket r, r + 2^{32-n} - 1 \rrbracket$, ensemble de cardinal 2^{32-n} . $\square$

Question 4. Le préfixe contient 2^{32-n} adresses ; deux sont réservées (réseau et diffusion), d'où

$$\#\{\text{adresses attribuables}\} = 2^{32-n} - 2 \quad (n \leq 30).$$

Pour $n = 31$ la formule donnerait 0 : un tel préfixe ne contient que deux adresses, toutes deux « réservées ». Cette situation est traitée à part ([RFC 3021](#)) : sur une liaison point-à-point il n'y a pas besoin d'adresse de diffusion, et les deux adresses sont attribuées aux deux extrémités. Pour $n = 32$ le préfixe se réduit à une adresse unique : c'est une *route d'hôte*. Ces deux cas montrent que la formule $2^{32-n} - 2$ est une convention d'ingénierie et non un théorème.

Question 5.

```

1 def dans_prefixe(a, p, n):
2     if n == 0:
3         return True
4     return (a ^ p) >> (32 - n) == 0

```

Démonstration. Notons $a_{31}, \dots, a_0$ et $p_{31}, \dots, p_0$ les bits de a et p . Le ou exclusif est bit à bit : le bit de rang i de $a \oplus p$ vaut 0 si et seulement si $a_i = p_i$. Le décalage $\gg(32 - n)$ ne conserve que les bits de rang $\geq 32 - n$, c'est-à-dire les n bits de poids fort. Donc

$$(a \oplus p) \gg (32 - n) = 0 \iff \forall i \geq 32 - n, a_i = p_i \iff a \in \mathcal{A}(p/n).$$

$\square$ Complexité $\Theta(1)$; on n'a besoin ni de calculer le masque ni de faire deux et logiques.

Question 6. (Agrégation.)

($\Leftarrow$) Supposons que p et q ne diffèrent que par le bit de rang $32 - n$; quitte à les échanger, $p < q$ et $q = p + 2^{32-n}$. D'après la question 3, $\mathcal{A}(p/n) = \llbracket p, p + 2^{32-n} - 1 \rrbracket$ et $\mathcal{A}(q/n) = \llbracket p + 2^{32-n}, p + 2^{32-n+1} - 1 \rrbracket$. Ces deux intervalles sont contigus, leur réunion est $\llbracket p, p + 2^{33-n} - 1 \rrbracket$. Or p a ses $32 - n$ bits de poids faible nuls et son bit de rang $32 - n$ nul (puisque $q = p + 2^{32-n}$ n'a pas provoqué de retenue), donc p a ses $33 - n$ bits de poids faible nuls : $p/(n - 1)$ est un préfixe valide et $\mathcal{A}(p/(n - 1)) = \llbracket p, p + 2^{33-n} - 1 \rrbracket$. On prend $r = p$.

($\Rightarrow$) Réciproquement, supposons $\mathcal{A}(r/(n - 1)) = \mathcal{A}(p/n) \cup \mathcal{A}(q/n)$. Le membre de gauche a pour cardinal 2^{33-n} et le membre de droite au plus $2 \cdot 2^{32-n} = 2^{33-n}$, avec égalité si et seulement si les deux préfixes sont disjoints. Ils le sont donc, et leur réunion est un intervalle : ils sont contigus, donc $q = p + 2^{32-n}$ (à échange près), ce qui signifie exactement que p et q ne diffèrent que par le bit de rang $32 - n$. $\square$

Intérêt. Un opérateur qui possède deux préfixes adjacents n'en annonce qu'un seul, deux fois plus grand : c'est l'agrégation CIDR, sans laquelle la table de routage globale d'Internet, aujourd'hui de l'ordre de 10^6 entrées, en compterait plusieurs dizaines de millions.

Question 7. (Famille laminaire.) Supposons $n \leq m$ (sinon on échange les rôles) et $\mathcal{A}(p/n) \cap \mathcal{A}(q/m) \neq \emptyset$; soit a dans l'intersection. Alors a coïncide avec p sur ses n bits de poids fort, et avec q sur ses $m \geq n$ bits de poids fort. Donc p et q coïncident sur leurs n bits de poids fort. Soit alors $b \in \mathcal{A}(q/m)$: b coïncide avec q sur $m \geq n$ bits, donc avec q — donc avec p — sur n bits, c'est-à-dire $b \in \mathcal{A}(p/n)$. Ainsi $\mathcal{A}(q/m) \subseteq \mathcal{A}(p/n)$.

Les trois cas sont donc exhaustifs ; ils sont mutuellement exclusifs dès que les deux préfixes sont distincts, puisque les ensembles sont non vides. $\square$

Autrement dit, l'ensemble des préfixes CIDR ordonné par inclusion est une forêt : c'est exactement ce qui permet la structure d'arbre de la question 20.

Question 8.

- (a) $\text{masque}(12) = 2^{32} - 2^{20}$. En base 256 : les 12 premiers bits sont à 1, soit $11111111.11110000.0.0$, c'est-à-dire $\boxed{255.240.0.0}$ (car $11110000_2 = 240$).
- (b) $172.16.35.7$ et le masque $255.240.0.0$: $172 \& 255 = 172$, $16 \& 240 = 16$ (car $16 = 00010000_2$), $35 \& 0 = 0$, $7 \& 0 = 0$. L'adresse de réseau est $\boxed{172.16.0.0}$.
- (c) $\text{masque}(26) = 255.255.255.192$ (car $192 = 11000000_2$). Or $130 = 10000010_2$, donc $130 \& 192 = 128$: le réseau est $\boxed{192.168.1.128}$. La diffusion s'obtient en mettant à 1 les 6 bits de poids faible : $128+63 = 191$, soit $\boxed{192.168.1.191}$. Le nombre d'adresses utilisables est $2^6 - 2 = \boxed{62}$.

Partie II — La somme de contrôle Internet

Question 9.

```

1 def add16(a, b):
2     s = a + b
3     return (s & 0xFFFF) + (s >> 16)

```

Démonstration. Soient $a, b \in \llbracket 0, N-1 \rrbracket$ avec $N = 2^{16}$. Alors $s = a + b \leq 2N - 2$, donc $\lfloor s/N \rfloor \in \{0, 1\}$; l'expression $(s \gg 16)$ vaut donc 0 ou 1, et le résultat est $(s \bmod N) + \lfloor s/N \rfloor$.

Cas $s < N$: le résultat vaut $s \leq N - 1$. Cas $s \geq N$: le résultat vaut $s - N + 1$, et $N \leq s \leq 2N - 2$ donne $1 \leq s - N + 1 \leq N - 1$.

Dans les deux cas le résultat est dans $\llbracket 0, N-1 \rrbracket$: **la réinjection de la retenue ne peut jamais provoquer une seconde retenue**. En effet, elle ne pourrait le faire que si $s \bmod N = N - 1$ et $\lfloor s/N \rfloor = 1$, c'est-à-dire $s = 2N - 1$, impossible puisque $s \leq 2N - 2$. $\square$

Question 10. Reprenons les deux cas. Si $s < N$, le résultat est $s = a + b$. Si $s \geq N$, le résultat est $s - (N - 1)$. Dans les deux cas

$$a \oplus' b \equiv a + b \pmod{N - 1}.$$

La commutativité est immédiate ($a + b = b + a$). Pour l'associativité, remarquons que $(a \oplus' b) \oplus' c$ et $a \oplus' (b \oplus' c)$ sont deux éléments de $\llbracket 0, N-1 \rrbracket$ congrus à $a + b + c$ modulo $N - 1$; deux tels éléments diffèrent d'un multiple de $N - 1$, donc sont égaux ou valent $\{0, N - 1\}$. Un examen des cas montre que $\oplus'$ ne produit 0 que si les deux opérandes sont nuls : si $s \geq N$ le résultat est ≥ 1 , et si $s < N$ le résultat est s , nul seulement si $a = b = 0$. Comme $a \oplus' 0 = a$, les deux parenthésages donnent 0 exactement dans le même cas ($a = b = c = 0$), et sont donc toujours égaux. $\square$

0 est neutre : $a \oplus' 0 = a$. Enfin $x \oplus' \bar{x} = x + (N - 1 - x) = N - 1 = 0\text{xFFFF}$.

Les deux zéros. 0x0000 et 0xFFFF sont tous deux congrus à 0 modulo $N - 1$: ce sont les deux représentations de zéro, notées $+0$ et -0 . L'opération $\oplus'$ ne produit jamais $+0$ à partir

d'opérandes non tous nuls : elle « préfère » -0 . Cette dissymétrie est *exactement* la source du bogu de la RFC 1141 étudié à la question 16.

Question 11. (a) Découpage en mots.

```

1 def mots16(octets):
2     o = list(octets) # copie : on ne modifie pas l'argument
3     if len(o) % 2 == 1:
4         o.append(0) # bourrage a droite
5     return [o[i]*256 + o[i+1] for i in range(0, len(o), 2)]

```

L'octet d'indice pair est celui de poids fort : c'est la convention « gros-boutienne » de l'ordre réseau. Sur l'exemple demandé, $\text{mots16}([0x45, 0x00, 0x00, 0x73]) = [0x4500, 0x0073]$.

On prendra garde à recopier la liste : sans le `list(octets)`, le bourrage modifierait la liste de l'appelant, effet de bord d'autant plus pernicieux qu'il ne se manifeste que pour les paquets de longueur impaire.

(b) Somme.

```

1 def somme_un(mots):
2     s = 0
3     for m in mots:
4         s = add16(s, m)
5     return s

```

L'accumulateur est initialisé à 0, élément neutre de $\oplus'$ (question 10).

(c) Complément.

```

1 def checksum(octets):
2     return somme_un(mots16(octets)) ^ 0xFFFF

```

Sur l'exemple : $0x4500 \oplus' 0x0073 = 0x4573$ (aucune retenue), d'où $\text{checksum} = \overline{0x4573} = 0xFFFF - 0x4573 = \boxed{0xBA8C}$.

Complexités. Notons k le nombre d'octets et $\ell = \lceil k/2 \rceil$ le nombre de mots.

	temps	espace auxiliaire
<code>mots16</code>	$\Theta(k)$	$\Theta(k)$ (la copie et la liste de mots)
<code>somme_un</code>	$\Theta(\ell) = \Theta(k)$	$\Theta(1)$ (un accumulateur)
<code>checksum</code>	$\Theta(k)$	$\Theta(k)$ (héritée de <code>mots16</code>)

Version en espace constant. La liste intermédiaire des mots ne sert à rien : on peut les former et les accumuler dans la même boucle.

```

1 def checksum_direct(octets):
2     s = 0
3     k = len(octets)
4     i = 0
5     while i + 1 < k: # paires completes
6         s = add16(s, octets[i]*256 + octets[i+1])
7         i = i + 2
8     if i < k: # dernier octet isole
9         s = add16(s, octets[i]*256) # bourrage implicite par un octet nul
10    return s ^ 0xFFFF

```

On obtient $\Theta(k)$ en temps et $\Theta(1)$ en espace auxiliaire, sans modifier l'argument. C'est exactement la boucle donnée en annexe du RFC 1071 ; sur un paquet de 1500 octets, l'économie de la liste intermédiaire est loin d'être négligeable lorsqu'on traite des millions de paquets par seconde.

Question 12. Notons S la somme en complément à un de tous les mots, le champ de contrôle étant à zéro. La valeur inscrite est $\overline{S}$. Le récepteur calcule la somme du paquet complet, c'est-à-dire, par associativité et commutativité (question 10) :

$$S \oplus' \overline{S} = N - 1 = 0xFFFF.$$

(On a utilisé $x \oplus' \bar{x} = 0\text{xFFFF}$, établi à la question 10.) □

```
1 def verifie(octets):
2     return somme_un(mots16(octets)) == 0xFFFF
```

On notera l'élégance du procédé : le récepteur n'a *pas* à extraire le champ de contrôle ni à le remettre à zéro. Il fait la même boucle que l'émetteur et compare à une constante.

Question 13. (Indépendance de l'ordre des octets.) Écrivons $x = 256u + v$ avec $u, v \in \llbracket 0, 255 \rrbracket$; alors $\sigma(x) = 256v + u$.

Il suffit de traiter le cas de deux mots, le cas général s'en déduisant par récurrence immédiate grâce à l'associativité. Soient $x = 256u + v$ et $y = 256u' + v'$.

Cas 1 : $v + v' \leq 255$ et $u + u' \leq 255$. Aucune retenue nulle part : $x \oplus' y = 256(u + u') + (v + v')$ et $\sigma(x) \oplus' \sigma(y) = 256(v + v') + (u + u')$, qui est bien son échangé.

Cas 2 : *retenue de l'octet de poids faible vers celui de poids fort.* Dans le calcul échangé, cette même retenue passe de l'octet de poids fort vers l'octet de poids faible *par la réinjection circulaire*. Le point clé, souligné par la [RFC 1071](#), est que l'addition en complément à un sur 16 bits est *circulaire* : les retenues vont du bit 7 au bit 8 et du bit 15 au bit 0. Échanger systématiquement les octets revient à tourner les bits de 8 positions dans ce cycle, ce qui ne modifie pas leur ordre relatif : la somme obtenue est la somme échangée. □

Conséquence pratique. Une machine « petit-boutienne » qui lit en mémoire, mot par mot, des données stockées en ordre réseau (« gros-boutien ») obtient automatiquement la somme échangée ; en réécrivant le résultat en mémoire, il est de nouveau échangé, donc correct. La somme de contrôle se calcule ainsi avec *exactement le même code* quelle que soit l'architecture — argument décisif en 1988.

Question 14. Supposons que le paquet reçu diffère du paquet émis par un seul bit, de rang $r \in \llbracket 0, 15 \rrbracket$ dans le mot d'indice i . La somme reçue S' vérifie $S' \equiv S \pm 2^r \pmod{N-1}$, où $S = 0\text{xFFFF} \equiv 0$. Or $1 \leq 2^r \leq 2^{15}$, donc $2^r \not\equiv 0 \pmod{N-1}$ (car $0 < 2^r < N-1$), et de même $-2^r \not\equiv 0$. Donc $S' \not\equiv 0$, c'est-à-dire $S' \neq 0\text{xFFFF}$ et $S' \neq 0\text{x0000}$: le test de la question 12 échoue. □

Vérification expérimentale : sur 3 000 paquets aléatoires de 20 octets et une inversion de bit tirée au hasard, l'erreur a été détectée dans 3 000 cas sur 3 000.

Question 15. L'addition $\oplus'$ étant **commutative**, permuter deux mots de 16 bits ne change pas la somme : une telle altération n'est *jamais* détectée. Par exemple, le paquet commençant par 12 34 AB CD et celui commençant par AB CD 12 34, tout le reste étant identique, ont la même somme de contrôle.

De même, une erreur de deux bits qui *se compense* passe inaperçue : ajouter 1 à un mot et retrancher 1 à un autre laisse la somme inchangée. Ainsi 12 35 AB CC a la même somme que 12 34 AB CD.

Comparaison avec un CRC. Un code polynomial interprète le message comme un polynôme sur $\mathbb{F}_2$ et en calcule le reste modulo un polynôme générateur : le résultat dépend de la *position* de chaque bit, ce qui détruit la symétrie ci-dessus. Un CRC-32 bien choisi détecte toutes les erreurs de 1, 2 ou 3 bits, toutes les rafales de longueur ≤ 32 et toutes les erreurs de poids impair. Il est en revanche plus coûteux à calculer et ne se met pas à jour incrémentalement (question 16). Le choix de la somme en complément à un pour la couche transport est donc un compromis assumé : la détection fine est confiée au CRC de la couche liaison, la somme de contrôle Internet n'assurant qu'un contrôle *de bout en bout* bon marché.

Question 16. (Mise à jour incrémentale.)

```
1 def maj_1141(HC, m, mp): # HC' = HC +' m +' ~m' (RFC 1141)
2     return add16(add16(HC, m), mp ^ 0xFFFF)
3
4 def maj_1624(HC, m, mp): # HC' = ~( ~HC +' ~m +' m' ) (RFC 1624)
5     return add16(add16(HC ^ 0xFFFF, m ^ 0xFFFF), mp) ^ 0xFFFF
```

(c) **Application numérique.** On a $C_{\text{autres}} = 0\text{xCD7A}$, $m = 0\text{x5555}$, $m' = 0\text{x3285}$. On note d'abord les deux sommes utiles :

$$0\text{xCD7A} + 0\text{x5555} = 0\text{x122CF} \longrightarrow 0\text{x22D0} \quad (\text{repli}), \quad 0\text{xCD7A} + 0\text{x3285} = 0\text{xFFFF}.$$

On en déduit :

$$\begin{aligned} HC &= \overline{0\text{x22D0}} = 0\text{xDD2F}, \\ HC' \text{ (recalcul complet)} &= \overline{0\text{xFFFF}} = \boxed{0\text{x0000}}, \\ HC' \text{ par la RFC 1141} &= 0\text{xDD2F} \oplus' 0\text{x5555} \oplus' \overline{0\text{x3285}} = 0\text{xFFFF} \quad \text{faux}, \\ HC' \text{ par la RFC 1624} &= \overline{0\text{x22D0} \oplus' \overline{0\text{x5555}} \oplus' 0\text{x3285}} = 0\text{x0000} \quad \text{correct.} \end{aligned}$$

(d) **Explication.** La formule de la RFC 1141 suppose que la complémentation est *distributive* sur $\oplus'$, c'est-à-dire $\overline{x \oplus' y} = \overline{x} \oplus' \overline{y}$; or cette propriété tombe en défaut lorsque le résultat est nul, précisément à cause des deux représentations de zéro (question 10). L'addition $\oplus'$ ne produit jamais $+0$ à partir d'opérandes non tous nuls : elle produit $-0 = 0\text{xFFFF}$. Comme l'en-tête contient toujours au moins un champ non nul, le champ de contrôle ne peut jamais valoir $\overline{+0} = 0\text{xFFFF}$; il peut en revanche valoir $\overline{-0} = 0\text{x0000}$. La RFC 1141 produit donc 0xFFFF , une valeur qu'un calcul complet ne peut jamais donner. La RFC 1624 évite le problème en n'utilisant la complémentation qu'une seule fois, tout à la fin.

Remarque. Le RFC 1624 note que si le récepteur vérifie comme à la question 12 (somme du paquet complet = 0xFFFF), les deux valeurs 0x0000 et 0xFFFF passent le test : le bogue n'a d'effet que sur les implantations qui recalculent la somme et la comparent au champ. C'est un bel exemple de bogue qui ne se manifeste qu'à l'interopérabilité.

Question 17. On regroupe les 20 octets en dix mots de 16 bits :

$$4500, 0073, 0000, 4000, 4011, 0000, \text{c0a8}, 0001, \text{c0a8}, 00\text{c7}.$$

La question 10 autorise à sommer d'abord *sans* se soucier des retenues, puis à les replier à la fin (c'est la technique dite des « retenues différées » du RFC 1071). La somme entière vaut

$$17664 + 115 + 0 + 16384 + 16401 + 0 + 49320 + 1 + 49320 + 199 = 149\,404 = 0\text{x2479C}.$$

On replie la retenue de poids fort : $0\text{x479C} + 0\text{x2} = 0\text{x479E}$, valeur qui tient sur 16 bits — aucun second repli n'est nécessaire. D'où

$$\boxed{\text{checksum} = \overline{0\text{x479E}} = 0\text{xFFFF} - 0\text{x479E} = 0\text{xB861}}$$

que l'on inscrit dans les octets d'indices 10 et 11 : `b8 61`.

Vérification. En recalculant la somme sur le paquet complet, on ajoute 0xB861 à 0x479E , ce qui donne exactement 0xFFFF : le test de la question 12 est satisfait. (C'est l'exemple classique du RFC 1071 ; le calcul a été vérifié en machine.)

Partie III — Routage : plus long préfixe et plus courts chemins

Question 18.

```

1 def plus_long_prefixe(table, a):
2     best, bl = None, -1
3     for (p, n, saut) in table:
4         if dans_prefixe(a, p, n) and n > bl:
5             bl, best = n, saut
6     return best

```

Chaque entrée est examinée une fois, en $O(1)$ (question 5) : la complexité est $\Theta(k)$.

Question 19. Soit $\mathcal{P}$ l'ensemble des préfixes de la table contenant a . Il est non vide (il contient la route par défaut). D'après la question 7, deux éléments de $\mathcal{P}$ sont comparables pour l'inclusion, puisque leur intersection contient a donc est non vide : $\mathcal{P}$ est **totalelement ordonné** par inclusion. Comme $\mathcal{A}(p/n) \subseteq \mathcal{A}(q/m) \Rightarrow n \geq m$, l'ordre par inclusion coïncide avec l'ordre inverse des longueurs : il existe donc un unique élément de longueur maximale, qui est aussi le plus petit pour l'inclusion. $\square$

C'est ce résultat qui donne son sens à la règle « plus long préfixe » : elle sélectionne la route la plus spécifique, et cette route est unique.

Question 20. On représente un nœud par une liste [fils0, fils1, saut].

```

1 class Trie:
2     def __init__(self):
3         self.racine = [None, None, None]
4
5     def inserer(self, p, n, saut):
6         noeud = self.racine
7         for i in range(n):
8             b = (p >> (31 - i)) & 1 # i-eme bit en partant du poids fort
9             if noeud[b] is None:
10                 noeud[b] = [None, None, None]
11             noeud = noeud[b]
12         noeud[2] = saut

```

Complexités. Une insertion effectuée exactement $n \leq 32$ tours en $O(1)$: elle coûte $O(n) = O(32) = O(1)$ si l'on considère 32 comme une constante. En espace, chaque insertion crée au plus 32 nœuds : la structure occupe $O(32k) = O(k)$ pour k entrées. Le facteur 32 est le prix de la structure, mais il se réduit fortement en pratique par compression des chaînes de nœuds à un seul fils (arbres *Patricia*).

Question 21.

```

1     def chercher(self, a):
2         noeud = self.racine
3         res = self.racine[2] # eventuelle route par défaut (/0)
4         for i in range(32):
5             b = (a >> (31 - i)) & 1
6             if noeud[b] is None:
7                 break
8             noeud = noeud[b]
9             if noeud[2] is not None:
10                 res = noeud[2]
11         return res

```

Invariant.

*Au début de l'itération i , **noeud** est le nœud de profondeur i correspondant aux i premiers bits de a (ce nœud existe), et **res** est le saut associé au plus long préfixe de longueur $\leq i$ de la table qui contienne a (**None** s'il n'y en a pas).*

Initialisation. Pour $i = 0$, **noeud** est la racine, qui représente la suite vide, et **res** est le saut du préfixe $/0$ s'il existe.

Hérédité. Supposons l'invariant vrai au début de l'itération i . Soit b le bit de rang $31 - i$ de a . Si **noeud**[b] est **None**, aucun préfixe de la table de longueur $> i$ ne peut contenir a (par construction de **inserer**, un tel préfixe aurait créé ce fils) : la boucle s'arrête et **res** est déjà la bonne valeur. Sinon on descend ; le nouveau nœud représente les $i + 1$ premiers bits de a . S'il porte un saut, c'est que la table contient le préfixe correspondant, de longueur $i + 1$, qui contient a et qui est plus long que tous les précédents : on met à jour **res**. L'invariant est rétabli pour $i + 1$.

Conclusion. À la sortie, `res` est le saut du plus long préfixe de la table contenant a , qui est unique d'après la question 19. $\square$

Vérification expérimentale : sur 300 tables aléatoires et 200 adresses par table, soit 60 000 recherches, l'arbre et la version exhaustive de la question 18 ont toujours donné le même résultat.

Question 22. La boucle fait au plus 32 tours en $O(1)$: la complexité est $O(32) = O(1)$, indépendante du nombre k d'entrées, contre $\Theta(k)$ pour la question 18.

Commentaire. Un routeur de cœur de réseau porte aujourd'hui une table d'environ 10^6 préfixes et doit décider du sort de plusieurs dizaines de millions de paquets par seconde. Le parcours exhaustif demanderait 10^6 tests par paquet — impossible. La recherche dans l'arbre en demande 32 au plus, soit un gain de plus de quatre ordres de grandeur. C'est exactement l'écart entre un algorithme linéaire et un algorithme logarithmique en la taille de l'espace d'adressage. En pratique les routeurs vont plus loin encore (arbres multi-bits, mémoires associatives), mais le principe reste celui-ci.

Question 23.

```

1 import heapq
2
3 def dijkstra(V, adj, s):
4     INF = float('inf')
5     d = [INF] * V
6     d[s] = 0
7     vus = [False] * V
8     tas = [(0, s)]
9     while tas:
10         du, u = heapq.heappop(tas)
11         if vus[u]:
12             continue # entree perimee
13         vus[u] = True
14         for (v, w) in adj[u]:
15             if d[u] + w < d[v]:
16                 d[v] = d[u] + w
17                 heapq.heappush(tas, (d[v], v))
18     return d

```

Complexité. Chaque arête provoque au plus un ajout dans le tas, qui contient donc au plus E éléments ; chaque opération de tas coûte $O(\log E) = O(\log V)$ (car $E \leq V^2$). D'où $O((V + E) \log V)$.

Le test `if vus[u]: continue` est indispensable : on n'enlève pas les entrées périmées du tas, on les ignore lorsqu'on les extrait. C'est la version « paresseuse », la plus simple à écrire correctement.

Question 24. Invariant de Dijkstra.

Au moment où un sommet u est marqué ($vus[u] = True$), $d[u]$ est la distance de s à u ; de plus, pour tout sommet v non encore marqué, $d[v]$ est le poids du plus court chemin de s à v dont tous les sommets intermédiaires sont marqués.

Démonstration de la première partie, par l'absurde. Supposons que u soit le premier sommet marqué pour lequel $d[u] > \delta(s, u)$, où δ désigne la vraie distance. Soit γ un plus court chemin de s à u , et soit y le premier sommet de γ non encore marqué au moment où l'on marque u (il en existe, car u l'est), et x son prédécesseur sur γ (x est marqué, et x existe car s est marqué en premier). Par minimalité de u , $d[x] = \delta(s, x)$; comme l'arête (x, y) a été relâchée lors du marquage de x , on a $d[y] \leq \delta(s, x) + w(x, y) = \delta(s, y)$. Or y est sur un plus court chemin vers u et les poids sont positifs, donc $\delta(s, y) \leq \delta(s, u)$. On obtient

$$d[y] \leq \delta(s, y) \leq \delta(s, u) < d[u],$$

ce qui contredit le fait que l'algorithme a extrait u — de clé minimale — avant y . $\square$

Où sert la positivité ? Exactement à l'inégalité $\delta(s, y) \leq \delta(s, u)$: un préfixe d'un plus court chemin n'est plus court que le chemin entier que si les poids sont ≥ 0 . Avec une arête de poids négatif, Dijkstra peut marquer définitivement un sommet à une valeur trop grande et ne jamais se corriger.

Question 25.

```

1 def bellman_ford(V, aretes, s):
2     INF = float('inf')
3     d = [INF] * V
4     d[s] = 0
5     for _ in range(V - 1):
6         change = False
7         for (u, v, w) in aretes:
8             if d[u] + w < d[v]:
9                 d[v] = d[u] + w
10                change = True
11         if not change:
12             break # arret anticipé
13     for (u, v, w) in aretes:
14         if d[u] + w < d[v]:
15             return None # circuit de poids négatif
16     return d

```

Complexité. $\Theta(V^2E)$ dans le pire des cas, $\Theta(E)$ en espace. C'est plus coûteux que Dijkstra, mais l'algorithme tolère les poids négatifs et — surtout — il est *réparti* : chaque sommet peut effectuer sa relaxation en ne connaissant que ses voisins. C'est ce qui en fait le cœur des protocoles à vecteur de distance.

Question 26. Invariant. Après i tours de la boucle externe, pour tout sommet v , $d[v] \leq \delta_i(s, v)$, où $\delta_i(s, v)$ est le poids minimal d'un chemin de s à v utilisant au plus i arêtes ($+\infty$ s'il n'en existe pas).

Initialisation. Pour $i = 0$: $\delta_0(s, s) = 0 = d[s]$ et $\delta_0(s, v) = +\infty$ pour $v \neq s$.

Hérédité. Soit γ un chemin de s à v à au plus $i + 1$ arêtes de poids $\delta_{i+1}(s, v)$. Si γ a au plus i arêtes, la conclusion découle de l'hypothèse de récurrence et du fait que $d[v]$ ne fait que décroître. Sinon, notons u l'avant-dernier sommet de γ : le préfixe de γ jusqu'à u a au plus i arêtes et est optimal parmi ceux-là, donc à la fin du tour i on a $d[u] \leq \delta_i(s, u)$. Lors du tour $i + 1$, l'arc (u, v, w) est relâché, d'où $d[v] \leq d[u] + w \leq \delta_i(s, u) + w = \delta_{i+1}(s, v)$. $\square$

Correction et nombre de tours. En l'absence de circuit de poids négatif, un plus court chemin est *élémentaire*, donc comporte au plus $V - 1$ arêtes : après $V - 1$ tours, $d[v] \leq \delta_{V-1}(s, v) = \delta(s, v)$. L'inégalité inverse est immédiate puisque toute valeur de $d[v]$ est le poids d'un chemin réel. D'où l'égalité. Enfin, si un tour supplémentaire produit encore une amélioration, c'est qu'il existe un chemin à V arêtes strictement meilleur que tout chemin à $V - 1$ arêtes : un tel chemin contient un circuit, nécessairement de poids strictement négatif.

Question 27. (Comptage à l'infini.) Sur la ligne $A - B - C$, à l'équilibre B annonce $d(C) = 1$ et A annonce $d(C) = 2$. Le lien $B - C$ tombe : B passe $d(C)$ à $+\infty$, mais il reçoit alors de A l'annonce « je sais atteindre C en 2 ». B en déduit $d(C) = 3$ (via A) et l'annonce. A recalcule alors $d(C) = 4$ (via B), etc.

Le mécanisme est clair : A a appris sa route *par* B mais l'annonce à B sans préciser cette provenance. Chaque routeur croit qu'un chemin subsiste chez l'autre ; les distances croissent d'un pas à chaque échange, indéfiniment. C'est le **comptage à l'infini**, défaut structurel des algorithmes à vecteur de distance — Bellman–Ford réparti converge, mais lentement, et transite par des états incohérents.

Contre-mesures. RIP en emploie deux.

- Le *split horizon* : un routeur n'annonce pas à un voisin une route apprise de ce voisin (variante

poison reverse : il l'annonce à distance infinie). Cela règle le cas à deux routeurs, mais pas les boucles plus longues.

- Un plafond : la distance 16 est déclarée infinie. Le comptage s'arrête donc en au plus 16 échanges — au prix d'une limitation drastique du diamètre du réseau, ce qui cantonne RIP aux petits domaines.

C'est précisément pour éviter ces pathologies qu'OSPF a adopté l'approche à *état de liens* : chaque routeur diffuse la topologie complète et exécute Dijkstra localement, au prix d'un coût mémoire et d'un trafic de diffusion supérieurs.

Question 28. On applique Dijkstra depuis le sommet 0. On note entre crochets le tableau d (avec ∞ pour l'infini) et l'on souligne le sommet extrait.

Étape	Sommet extrait	Tableau $d = [d_0, \dots, d_5]$ après relaxation
0	—	$[0, \infty, \infty, \infty, \infty, \infty]$
1	0	$[0, 7, 9, \infty, \infty, 14]$
2	1 ($d = 7$)	$[0, 7, 9, 22, \infty, 14]$
3	2 ($d = 9$)	$[0, 7, 9, 20, \infty, 11]$
4	5 ($d = 11$)	$[0, 7, 9, 20, 20, 11]$
5	3 ($d = 20$)	$[0, 7, 9, 20, 20, 11]$
6	4 ($d = 20$)	$[0, 7, 9, 20, 20, 11]$

D'où $d = [0, 7, 9, 20, 20, 11]$.

Un plus court chemin de 0 à 4 se lit en remontant les prédécesseurs : 4 a été atteint depuis 5 ($11 + 9 = 20$), 5 depuis 2 ($9 + 2 = 11$), 2 depuis 0 (9). D'où

$$0 \rightarrow 2 \rightarrow 5 \rightarrow 4 \quad \text{de poids } 9 + 2 + 9 = 20.$$

On notera que le chemin direct $0 \rightarrow 5$ coûte 14 alors que le détour $0 \rightarrow 2 \rightarrow 5$ ne coûte que 11 : l'arête la plus « courte » n'est pas toujours la bonne, ce qui justifie l'algorithme. Bellman-Ford donne évidemment le même tableau.

Partie IV — TCP : fiabilité, temporisateur et congestion

Question 29.

```

1 M = 1 << 32
2
3 def avant(a, b):
4     return ((b - a) % M) < (1 << 31) and a != b

```

Démonstration. Supposons que tous les numéros en circulation appartiennent à $\{c, c+1, \dots, c+W-1\}$ modulo M , avec $W < 2^{31}$. Soient $a = c+i$ et $b = c+j$ deux d'entre eux, $0 \leq i, j \leq W-1$, $i \neq j$. Alors $(b-a) \bmod M = (j-i) \bmod M$ avec $-(W-1) \leq j-i \leq W-1$.

- Si $j > i$ (c'est-à-dire a précède b), alors $0 < j-i \leq W-1 < 2^{31}$ et $(b-a) \bmod M = j-i < 2^{31}$: la fonction renvoie **True**.
- Si $j < i$, alors $(b-a) \bmod M = M + (j-i) \geq M - (W-1) > M - 2^{31} = 2^{31}$: la fonction renvoie **False**.

Le verdict est donc correct. $\square$

Ambiguïté pour $W = 2^{31}$. Prenons $a = 0$ et $b = 2^{31}$. Alors $(b-a) \bmod M = 2^{31} \not< 2^{31}$: la fonction déclare que a ne précède pas b . Mais $(a-b) \bmod M = 2^{31}$ également : elle déclare aussi que b ne précède pas a . Les deux numéros sont à distance égale dans les deux sens sur le cercle : la question « lequel vient avant ? » n'a tout simplement pas de réponse. C'est pourquoi TCP impose une fenêtre strictement inférieure à 2^{31} octets.

Question 30.

```

1 def fusionner(segments):
2     L = sorted(segments) # tri lexicographique : par debut
3     res = []
4     for (d, f) in L:
5         if res and d <= res[-1][1]: # chevauchement ou contiguite
6             res[-1] = (res[-1][0], max(res[-1][1], f))
7         else:
8             res.append((d, f))
9     return res

```

Complexité. $\Theta(k \log k)$ pour le tri, puis $\Theta(k)$ pour le balayage : total $\Theta(k \log k)$. C'est optimal dans le modèle des comparaisons, puisque le problème permet de trier (k intervalles disjoints en entrée ressortent triés).

Question 31. Invariant.

Avant le traitement de $L[i]$, la liste res contient les intervalles maximaux de la réunion $\bigcup_{j < i} L[j]$, triés par début croissant, deux à deux disjoints et non adjacents ; de plus, le début de $L[i]$ est supérieur ou égal au début du dernier élément de res .

Initialisation. Pour $i = 0$, res est vide : trivial.

Hérédité. Soit $(d, f) = L[i]$ et (d_0, f_0) le dernier élément de res . Comme L est trié par début croissant, $d \geq d_0$, et tout intervalle de res autre que le dernier se termine avant $d_0 \leq d$: seul le dernier peut rencontrer (d, f) . Deux cas.

- Si $d \leq f_0$, les deux intervalles se rencontrent ou sont contigus ; comme $d_0 \leq d$, leur réunion est exactement $[d_0, \max(f_0, f)]$, ce qu'affecte l'algorithme. Le nombre d'intervalles ne change pas, l'invariant est préservé.
- Si $d > f_0$, l'intervalle (d, f) est disjoint de tous les précédents *et* non adjacent au dernier ; on l'ajoute en queue, ce qui préserve le tri et la disjonction.

□

Conclusion. À la fin, res est la décomposition en intervalles maximaux de la réunion de tous les segments. Cette décomposition est **unique** : c'est la partition en composantes connexes de la réunion, vue comme partie de $\mathbb{Z}$.

Vérification expérimentale : sur 2000 jeux de segments aléatoires, la réunion des intervalles renvoyés a toujours coïncidé, ensemble d'entiers par ensemble d'entiers, avec la réunion des segments fournis, et la sortie était toujours strictement croissante.

Question 32.

```

1 def rto_initial(R, G=0.001):
2     SRTT = R
3     RTTVAR = R / 2
4     return SRTT, RTTVAR, max(1.0, SRTT + max(G, 4 * RTTVAR))
5
6 def rto_suivant(SRTT, RTTVAR, R, alpha=1/8, beta=1/4, G=0.001):
7     RTTVAR = (1 - beta) * RTTVAR + beta * abs(SRTT - R) # D'ABORD la variance
8     SRTT = (1 - alpha) * SRTT + alpha * R # ENSUITE la moyenne
9     return SRTT, RTTVAR, max(1.0, SRTT + max(G, 4 * RTTVAR))

```

Pourquoi cet ordre ? La mise à jour de RTTVAR utilise l'écart $|\text{SRTT} - R'|$ entre l'ancienne estimation et la nouvelle mesure. Si l'on mettait d'abord SRTT à jour, cet écart serait calculé avec une moyenne déjà attirée vers R' , donc systématiquement sous-évalué : l'estimateur de dispersion serait biaisé vers le bas, le RTO trop court, et les retransmissions inutiles se multiplieraient. Le RFC 6298 écrit d'ailleurs explicitement que l'ordre *doit* être respecté.

Question 33. Démonstration par récurrence sur n . La relation est $S_n = (1 - \alpha)S_{n-1} + \alpha R_n$ pour $n \geq 1$, avec $S_0 = R_0$.

Initialisation ($n = 0$). La formule donne $(1 - \alpha)^0 R_0 = R_0 = S_0$, la somme étant vide.

Hérédité. Supposons la formule vraie au rang n . Alors

$$\begin{aligned} S_{n+1} &= (1 - \alpha)S_n + \alpha R_{n+1} = (1 - \alpha) \left[(1 - \alpha)^n R_0 + \alpha \sum_{i=1}^n (1 - \alpha)^{n-i} R_i \right] + \alpha R_{n+1} \\ &= (1 - \alpha)^{n+1} R_0 + \alpha \sum_{i=1}^n (1 - \alpha)^{n+1-i} R_i + \alpha (1 - \alpha)^0 R_{n+1} = (1 - \alpha)^{n+1} R_0 + \alpha \sum_{i=1}^{n+1} (1 - \alpha)^{n+1-i} R_i. \end{aligned}$$

□

Somme des coefficients. En posant $j = n - i$, la somme géométrique donne

$$(1 - \alpha)^n + \alpha \sum_{i=1}^n (1 - \alpha)^{n-i} = (1 - \alpha)^n + \alpha \sum_{j=0}^{n-1} (1 - \alpha)^j = (1 - \alpha)^n + \alpha \cdot \frac{1 - (1 - \alpha)^n}{\alpha} = 1.$$

S_n est donc bien une **moyenne pondérée** des mesures, les poids décroissant géométriquement vers le passé : c'est une moyenne mobile exponentielle. Cela garantit en particulier $\min_i R_i \leq S_n \leq \max_i R_i$.

Vérification : sur 500 suites aléatoires, la forme close et l'itération coïncident à 10^{-12} près.

Question 34. Les poids de la question 33, lus du plus récent au plus ancien, sont α , $\alpha(1 - \alpha)$, $\alpha(1 - \alpha)^2, \dots$. On calcule, pour $|1 - \alpha| < 1$,

$$\mu = \sum_{j \geq 0} (j + 1) \alpha (1 - \alpha)^j = \alpha \sum_{j \geq 0} (j + 1) x^j \Big|_{x=1-\alpha} = \frac{\alpha}{(1 - x)^2} \Big|_{x=1-\alpha} = \frac{\alpha}{\alpha^2} = \boxed{\frac{1}{\alpha}}$$

(en dérivant terme à terme la série géométrique $\sum x^j = \frac{1}{1-x}$, licite sur $] -1, 1[$). Pour $\alpha = 1/8$: $\mu = \boxed{8}$ échantillons.

Pour le second point, on cherche le plus petit j tel que $\alpha(1 - \alpha)^j < \frac{1}{10}\alpha$, soit $(7/8)^j < 1/10$, c'est-à-dire $j > \ln 10 / \ln(8/7) \approx 2,3026/0,1335 \approx 17,24$. D'où $\boxed{j = 18}$.

Interprétation. L'estimateur « se souvient » d'environ 8 mesures et a pratiquement oublié tout ce qui date de plus de 18 aller-retours. C'est un compromis assumé : assez de mémoire pour lisser le bruit, assez peu pour suivre un changement de route ou de charge en quelques secondes.

Question 35. Algorithme de Karn. Un segment retransmis donne lieu à *deux* émissions et à un seul accusé de réception, sans que l'on sache à laquelle il répond. Si l'on mesure R depuis la *seconde* émission alors que l'accusé répondait à la première, on obtient une valeur beaucoup trop **petite**. L'estimateur diminue, le RTO diminue, donc davantage de segments expirent et sont retransmis — d'où de nouvelles sous-estimations : le biais s'auto-entretient et peut provoquer un effondrement par congestion. Karn et Partridge (1987) imposent donc de ne jamais échantillonner sur un segment retransmis, sauf si l'estampille temporelle lève l'ambiguïté.

Repli exponentiel. Après n expirations, les attentes successives valent $T_0, 2T_0, \dots, 2^{n-1}T_0$, d'où le temps cumulé

$$\sum_{i=0}^{n-1} 2^i T_0 = T_0 (2^n - 1).$$

Pour $T_0 = 1$ s et $n = 5$: $\boxed{31 \text{ secondes}}$. Ce doublement est la réponse canonique à une congestion : en cas de doute, l'émetteur se retire exponentiellement du réseau plutôt que d'insister.

Question 36. Les transmissions étant indépendantes et chacune réussissant avec probabilité $1 - p$, N est le rang du premier succès dans une suite d'épreuves de Bernoulli : N suit une **loi géométrique** de paramètre $1 - p$, à valeurs dans $\mathbb{N}^*$:

$$\mathbb{P}(N = k) = p^{k-1}(1 - p), \quad \mathbb{P}(N > k) = p^k, \quad \mathbb{E}[N] = \frac{1}{1 - p}.$$

($\mathbb{P}(N > k)$ s'obtient directement : les k premières transmissions échouent, ce qui a probabilité p^k par indépendance.)

Terminaison presque sûre. L'événement « ne jamais aboutir » est l'intersection décroissante des $\{N > k\}$, donc par continuité décroissante

$$\mathbb{P}(N = +\infty) = \lim_{k \rightarrow +\infty} p^k = 0 \quad \text{car } 0 < p < 1.$$

□

Pour $p = 0,1$: $\mathbb{E}[N] = 1/0,9 \approx \boxed{1,11}$ transmission, et $\mathbb{P}(N > 3) = 10^{-3}$. Le pire des cas n'est en revanche pas borné : c'est un algorithme de type Las Vegas, dont on ne peut analyser que le comportement moyen.

Question 37. Pendant un cycle, la fenêtre prend successivement les valeurs $\frac{W_{\max}}{2}, \frac{W_{\max}}{2} + 1, \dots, W_{\max} - 1$, à raison d'un aller-retour par valeur ; le cycle dure donc $W_{\max}/2$ aller-retours. En posant $W = W_{\max}$, le nombre de segments émis est

$$N_{\text{cycle}} = \sum_{i=W/2}^{W-1} i = \sum_{i=0}^{W-1} i - \sum_{i=0}^{W/2-1} i = \frac{(W-1)W}{2} - \frac{(W/2-1)(W/2)}{2},$$

soit, en développant,

$$N_{\text{cycle}} = \frac{W^2 - W}{2} - \frac{W^2/4 - W/2}{2} = \boxed{\frac{3W^2}{8} - \frac{W}{4}}.$$

La fenêtre moyenne est le quotient par le nombre d'aller-retours :

$$\bar{W} = \frac{N_{\text{cycle}}}{W/2} = \frac{2}{W} \left(\frac{3W^2}{8} - \frac{W}{4} \right) = \frac{3W}{4} - \frac{1}{2} = \boxed{\frac{3W-2}{4}}.$$

□

Vérification : pour $W = 8, 16, 64, 100$ on obtient respectivement $N_{\text{cycle}} = 22, 92, 1520, 3725$ et $\bar{W} = 5,5 ; 11,5 ; 47,5 ; 74,5$, en accord exact avec les formules.

Question 38. Une perte par cycle et une probabilité de perte p par segment donnent $p = 1/N_{\text{cycle}}$, soit

$$\frac{1}{p} = \frac{3W^2}{8} - \frac{W}{4} \sim \frac{3W^2}{8} \quad \text{quand } W \rightarrow +\infty,$$

d'où $W \approx \sqrt{\frac{8}{3p}}$. Le débit est le nombre d'octets émis par unité de temps, c'est-à-dire $\bar{W} \cdot \text{MSS} / \text{RTT}$ avec $\bar{W} \sim \frac{3}{4}W$:

$$\text{débit} \approx \frac{3}{4} \sqrt{\frac{8}{3p}} \cdot \frac{\text{MSS}}{\text{RTT}} = \frac{\text{MSS}}{\text{RTT}} \sqrt{\frac{9}{16} \cdot \frac{8}{3p}} = \frac{\text{MSS}}{\text{RTT}} \sqrt{\frac{3}{2p}} \approx \frac{1,2247 \text{ MSS}}{\text{RTT} \sqrt{p}}.$$

□

C'est la **formule de Mathis** (1997), l'un des résultats les plus cités de la littérature réseau : elle donne, à partir de trois paramètres seulement, le débit d'une connexion TCP en régime établi.

Question 39. Avec $\text{MSS} = 1460$ octets et $\text{RTT} = 0,100$ s :

p	$W_{\max} \approx \sqrt{8/(3p)}$	débit (octets/s)	débit (Mbit/s)
10^{-2}	16,3	$1,79 \times 10^5$	1,43
10^{-4}	163,3	$1,79 \times 10^6$	14,3

Question 40. (a) Le débit varie comme $p^{-1/2}$: diviser le taux de perte par 4 ne multiplie le débit que par 2. Les rendements sont donc *décroissants* ; améliorer la qualité d'un lien déjà bon rapporte peu. C'est un argument classique en faveur des mécanismes de retransmission locale sur les liens sans fil.

(b) Le débit varie comme $1/\text{RTT}$: à taux de perte égal, une connexion de RTT deux fois plus court obtient un débit deux fois plus grand. Deux connexions partageant le même goulet d'étranglement ne se partagent donc **pas** équitablement la bande passante : la plus proche écrase la plus lointaine. C'est le fameux biais de TCP contre les connexions à long délai, l'une des principales motivations des algorithmes de congestion modernes (CUBIC, dont la croissance ne dépend plus du RTT ; BBR, qui estime directement le débit et le délai plutôt que d'inférer la congestion des pertes).

Partie V — HTTP, cache et HTTPS

Question 41.

```

1 def analyser_requete(texte):
2     i = texte.find('\r\n\r\n')
3     if i < 0:
4         return None # en-tete incomplet
5     lignes = texte[:i].split('\r\n')
6     methode, cible, version = lignes[0].split(' ')
7     entetes = {}
8     for ligne in lignes[1:]:
9         j = ligne.index(':')
10        entetes[ligne[:j].lower()] = ligne[j+1:].strip()
11    return (methode, cible, version, entetes, texte[i+4:])

```

Complexité $\Theta(n)$ où n est la longueur du texte : la recherche, la découpe et le parcours des lignes sont linéaires. La mise en minuscules des noms est imposée par le [RFC 9110](#) : les noms d'en-têtes sont insensibles à la casse.

Sur l'exemple du sujet, on obtient

```

1 ('GET', '/index.html?p=1', 'HTTP/1.1',
2  {'host': 'cpgeacademy.org', 'content-length': '5'}, 'HELLO')

```

Question 42. On cherche la première occurrence du mot $u = \text{CRLF}\text{CRLF}$ de longueur 4. L'automate a pour états $\{0, 1, 2, 3\}$, l'état q signifiant « les q derniers caractères lus forment le préfixe de longueur q de u » (4 est l'état d'acceptation, où l'on s'arrête).

état	'\r'	'\n'	autre	signification
0	1	0	0	rien de reconnu
1	1	2	0	CR lu
2	3	0	0	CRLF lu
3	1	accepter	0	CRLFCR lu

La seule subtilité est la transition $1 \xrightarrow{\text{CR}} 1$ et $3 \xrightarrow{\text{CR}} 1$: après un CR inattendu, ce CR peut lui-même amorcer une nouvelle occurrence, il ne faut donc pas retomber en 0.

```

1 def fin_entete(flux):
2     etat = 0
3     for k, c in enumerate(flux):
4         if etat == 0: etat = 1 if c == '\r' else 0
5         elif etat == 1: etat = 2 if c == '\n' else (1 if c == '\r' else 0)
6         elif etat == 2: etat = 3 if c == '\r' else 0
7         elif etat == 3:
8             if c == '\n':

```

```

9         return k + 1
10        etat = 1 if c == '\r' else 0
11    return -1

```

Question 43. Invariant.

Après la lecture des k premiers caractères sans acceptation, *etat* est la longueur du plus long suffixe du préfixe lu qui soit un préfixe strict de $u = \text{CRLF}\text{CRLF}$.

Initialisation. Avant toute lecture, le seul suffixe est le mot vide : *etat* = 0.

Hérédité. Soit q l'état courant et c le caractère lu. Le plus long suffixe du nouveau texte qui soit préfixe de u est nécessairement de longueur $\leq q+1$. S'il est de longueur $q+1$, c'est que $u_q = c$: c'est exactement la transition « avancer ». Sinon, il faut prendre le plus long *bord* de $u_{0..q-1}$ suivi de c . Ici $u = \text{CRLF}\text{CRLF}$ et $u_{0..2} = \text{CRLF}$ n'a pour bords que le mot vide, sauf $u_{0..0} = \text{CR}$ dont le seul bord est vide : en pratique, si $c = \text{CR}$ on retombe en 1, sinon en 0. Ce sont précisément les transitions écrites. $\square$

Conséquences. Chaque caractère est lu **une seule fois** et traité en $O(1)$: l'analyse est en $\Theta(n)$, sans retour en arrière ni mémoire tampon. C'est essentiel pour un serveur qui traite un flux au fil de l'eau, sans pouvoir revenir sur les octets déjà consommés.

Vérification sur `"\r\n\r\n\r\n"` (longueur 7) :

k	début	0	1	2	3	4	5	6
caractère	—	CR	LF	CR	CR	LF	CR	LF
état après	0	1	2	3	1	2	3	accept.

La fonction renvoie 7. On vérifie l'invariant au rang $k = 3$: le texte lu est `CRLF`CR, dont le plus long suffixe préfixe de u après lecture du second CR est CR, de longueur 1 — c'est bien l'état obtenu. Une implantation naïve qui serait retombée en 0 aurait manqué l'occurrence.

Question 44.

```

1 def decoder_chunked(s):
2     res = []
3     i = 0
4     while True:
5         j = s.index('\r\n', i) # fin de la ligne de taille
6         n = int(s[i:j], 16) # taille en hexadecimal
7         if n == 0:
8             return ''.join(res)
9         res.append(s[j+2 : j+2+n])
10        i = j + 2 + n + 2 # on saute le bloc et son CRLF

```

Variante. Posons $V = \text{len}(s) - i$. À chaque tour, i augmente d'au moins $\text{len}(\text{hex}) + 2 + n + 2 \geq 5 > 0$: V est un entier strictement décroissant, minoré par 0 tant que la chaîne est bien formée. La boucle se termine donc, au plus tard en épuisant la chaîne (ce qui lèverait alors une exception, comportement attendu sur une entrée malformée).

Complexité. $\Theta(n)$: chaque caractère est examiné au plus deux fois (une par `index`, une par la copie), et la concaténation finale par `join` est linéaire. On évitera soigneusement `res = res + bloc` dans la boucle, qui donnerait $\Theta(n^2)$.

Exemple : le corps `"4\r\nWiki\r\n5\r\npedia\r\nE\r\n in\r\n\r\nchunks.\r\n0\r\n\r\n"` se décode en `"Wikipedia in\r\n\r\nchunks."` — noter que le troisième bloc, de taille $E_{16} = 14$, contient lui-même des CRLF : c'est tout l'intérêt du préfixe de longueur.

Question 45. La difficulté est d'obtenir $O(1)$ à la fois pour la recherche et pour la mise à jour de l'ordre d'ancienneté. On combine donc :

- un **dictionnaire** clé $\mapsto$ nœud, pour la recherche en $O(1)$ moyen ;

- une **liste doublement chaînée** des clés, de la plus récemment utilisée (en tête) à la plus ancienne (en queue), avec deux nœuds sentinelles pour éviter les cas particuliers.

Un nœud est une liste `[cle, precedent, suivant]`.

```

1 class LRU:
2     def __init__(self, cap):
3         self.cap = cap
4         self.d = {}
5         self.tete = ['GARDE', None, None] # sentinelles
6         self.queue = ['GARDE', self.tete, None]
7         self.tete[2] = self.queue
8
9     def _retirer(self, x):
10        x[1][2] = x[2]
11        x[2][1] = x[1]
12
13    def _en_tete(self, x):
14        x[1] = self.tete
15        x[2] = self.tete[2]
16        self.tete[2][1] = x
17        self.tete[2] = x
18
19    def acceder(self, cle):
20        if cle in self.d: # succes
21            x = self.d[cle]
22            self._retirer(x)
23            self._en_tete(x)
24            return True
25        if len(self.d) == self.cap: # eviction du plus ancien
26            v = self.queue[1]
27            self._retirer(v)
28            del self.d[v[0]]
29            x = [cle, None, None]
30            self._en_tete(x)
31            self.d[cle] = x
32            return False

```

Invariant.

À tout instant, la liste chaînée contient exactement les clés du dictionnaire, dans l'ordre décroissant de la date de leur dernier accès ; le dictionnaire associe à chaque clé le nœud qui la porte ; et le nombre de clés n'excède jamais `cap`.

Chaque opération de `acceder` — test d'appartenance, retrait, insertion en tête, suppression dans le dictionnaire — se fait en $O(1)$ moyen, le retrait en $O(1)$ exact grâce au *double* chaînage (sans lui, retirer un nœud demanderait de retrouver son prédécesseur, en $O(C)$). D'où un accès en $O(1)$ en moyenne, et un espace $O(C)$.

Vérification expérimentale : sur 3 000 suites d'accès aléatoires et des capacités de 1 à 5, cette classe et une implantation naïve de référence (liste Python réordonnée à chaque accès) ont toujours donné le même nombre de succès.

Question 46.

Accès	Résultat	Cache après (du plus récent au plus ancien)	Éviction
a	échec	$[a]$	—
b	échec	$[b, a]$	—
c	échec	$[c, b, a]$	—
a	succès	$[a, c, b]$	—
d	échec	$[d, a, c]$	b
b	échec	$[b, d, a]$	c
a	succès	$[a, b, d]$	—
c	échec	$[c, a, b]$	d
d	échec	$[d, c, a]$	b
a	succès	$[a, d, c]$	—

On compte 3 succès sur 10, soit un taux de succès de 30 %.

Commentaire. L'objet a , régulièrement redemandé, est toujours conservé : c'est le comportement attendu. En revanche b , c et d s'évincent mutuellement en boucle. La politique optimale hors ligne (algorithme de Belady, qui évince l'objet dont la prochaine demande est la plus lointaine) ferait mieux ; mais elle suppose de connaître l'avenir, ce qui est exclu en ligne.

Question 47. Initialisation. $m_0(j) = 0$ pour tout j : sans objet, on ne sert rien.

Récurrence. Un sous-ensemble optimal de $\{0, \dots, i\}$ de taille $\leq j$ contient ou non l'objet i :

$$m_{i+1}(j) = \begin{cases} m_i(j) & \text{si } j < t_i, \\ \max(m_i(j), m_i(j - t_i) + f_i) & \text{si } j \geq t_i. \end{cases}$$

Dans le second cas, le premier terme correspond au choix « ne pas prendre i », le second à « prendre i », ce qui laisse une capacité $j - t_i$ pour les objets $0, \dots, i - 1$. C'est bien une partition des solutions, d'où le maximum.

Question 48.

```

1 def cache_optimal(t, f, C):
2     m = [0] * (C + 1)
3     for i in range(len(t)):
4         for j in range(C, t[i] - 1, -1): # parcours DECROISSANT
5             if m[j - t[i]] + f[i] > m[j]:
6                 m[j] = m[j - t[i]] + f[i]
7     return m[C]
```

Sens de parcours. La case j est calculée à partir de la case $j - t_i < j$. En parcourant j de C vers t_i , la case $j - t_i$ n'a pas encore été modifiée au cours de cette itération : elle contient donc $m_i(j - t_i)$, valeur qui *n'utilise pas* l'objet i . L'objet est ainsi pris au plus une fois.

Contre-exemple en parcours croissant. Prenons un seul objet, $t = [3]$, $f = [10]$, et $C = 9$. En croissant : $j = 3$ donne $m[3] = 10$; puis $j = 6$ lit $m[3] = 10$ et donne $m[6] = 20$; puis $j = 9$ lit $m[6]$ et donne $m[9] = 30$. L'algorithme renverrait 30 en plaçant *trois fois* le même objet dans le cache, alors que la réponse correcte est 10. (Le parcours croissant résout en fait le sac à dos *avec répétition*, un autre problème.)

Question 49. Invariant.

À la fin de la i -ième itération de la boucle externe, pour tout $j \in \llbracket 0, C \rrbracket$, $m[j] = m_i(j)$, c'est-à-dire le nombre maximal de requêtes servies par un sous-ensemble de $\{0, \dots, i-1\}$ de taille totale au plus j .

Initialisation. Avant la première itération, m est nul, c'est m_0 .

Hérédité. Supposons $m = m_i$ au début de l'itération traitant l'objet i . Notons m^{fin} le tableau à la fin de cette itération.

($\leq$) Toute valeur écrite dans $m^{\text{fin}}[j]$ vaut $m_i(j - t_i) + f_i$, donc est réalisée par un sous-ensemble de $\{0, \dots, i-1\}$ de taille $\leq j - t_i$ auquel on ajoute i : c'est un sous-ensemble de $\{0, \dots, i\}$ de taille $\leq j$. Les valeurs conservées sont les $m_i(j)$, également réalisables. Donc $m^{\text{fin}}[j] \leq m_{i+1}(j)$.

($\geq$) Soit $S \subseteq \{0, \dots, i\}$ de taille $\leq j$ réalisant $m_{i+1}(j)$. Si $i \notin S$ alors $\sum_{k \in S} f_k \leq m_i(j) \leq m^{\text{fin}}[j]$, puisque le tableau n'est jamais diminué. Si $i \in S$, alors $S \setminus \{i\}$ est de taille $\leq j - t_i$, donc $\sum_{k \in S} f_k \leq m_i(j - t_i) + f_i$; or, grâce au parcours décroissant, la case $j - t_i$ contenait bien $m_i(j - t_i)$ au moment de la lecture, et la mise à jour a porté la case j à au moins cette valeur. Donc $m^{\text{fin}}[j] \geq m_{i+1}(j)$.

D'où l'égalité, et l'invariant. $\square$

Terminaison. Les deux boucles sont des boucles **for** sur des intervalles finis : la terminaison est immédiate, le nombre total d'itérations étant au plus $n(C + 1)$.

Conclusion. À la fin, $m[C] = m_n(C) = \mu(t, f, C)$.

Question 50. Complexité. Temps $\Theta(nC)$; espace $\Theta(C)$ auxiliaire, contre $\Theta(nC)$ si l'on conservait tous les m_i .

Appartenance à NP. Un certificat de l'instance « existe-t-il S avec $\sum_{i \in S} t_i \leq C$ et $\sum_{i \in S} f_i \geq F$? » est la liste des indices de S , de taille $O(n \log n)$ bits donc polynomiale. Le vérificateur calcule les deux sommes ($2n$ additions d'entiers de taille au plus celle de l'entrée) et effectue deux comparaisons : il est polynomial. Donc le problème est dans **NP**. $\square$

Pourquoi il n'y a pas contradiction. La taille de l'entrée n'est pas C mais le nombre de bits nécessaires pour l'écrire, soit $\Theta(\log C)$. La complexité $\Theta(nC) = \Theta(n 2^{\log_2 C})$ est donc *exponentielle* en la taille de l'entrée. On dit l'algorithme **pseudo-polynomial** : polynomial en la *valeur* des données, non en leur *taille*.

Tailles en octets, $C = 10$ Go. On aurait $C \approx 10^{10}$: le tableau à lui seul occuperait des dizaines de gigaoctets et le calcul demanderait $n \cdot 10^{10}$ opérations — irréalisable. On change donc d'unité (par exemple le mégaoctet, ou le PGCD des tailles), ce qui divise C par autant, ou l'on renonce à l'optimum exact au profit de l'heuristique gloutonne consistant à trier les objets par densité f_i/t_i décroissante — laquelle donne, en version fractionnaire, une 2-approximation.

Question 51. Avec $t = [3, 4, 5, 9, 4]$, $f = [60, 50, 70, 90, 30]$, $C = 10$:

$$\boxed{\mu = 130}, \quad \text{atteint par } S^* = \{0, 2\} \quad (\text{tailles } 3 + 5 = 8 \leq 10, \text{ fréquences } 60 + 70 = 130).$$

C'est le *seul* sous-ensemble optimal. On remarquera que l'objet 3, de fréquence maximale 90, n'est pas retenu : sa taille 9 mobiliserait presque tout le cache pour un gain moindre. C'est exactement ce que le glouton par fréquence décroissante manquerait (90 puis plus rien, soit $90 < 130$) ; le glouton par densité f_i/t_i , lui, donne $70/5 = 14$, $60/3 = 20$, $50/4 = 12,5$, $30/4 = 7,5$, $90/9 = 10$, soit l'ordre 0, 2, ... et retrouve ici l'optimum — mais ce n'est pas garanti en général.

Déroulé pour $t = [3, 4, 5]$, $f = [60, 50, 70]$, $C = 8$:

j	0	1	2	3	4	5	6	7	8
initialisation	0	0	0	0	0	0	0	0	0
après $(t_0, f_0) = (3, 60)$	0	0	0	60	60	60	60	60	60
après $(t_1, f_1) = (4, 50)$	0	0	0	60	60	60	60	110	110
après $(t_2, f_2) = (5, 70)$	0	0	0	60	60	70	70	110	130

D'où $\mu([3, 4, 5], [60, 50, 70], 8) = \boxed{130}$, réalisé par $\{0, 2\}$ (taille 8).

Requêtes SQL

Question 52.

```

1 SELECT methode, COUNT(*) AS nb, SUM(octets) AS total_octets
2 FROM journal
3 GROUP BY methode
4 ORDER BY nb DESC;
```

Question 53.

```

1 SELECT chemin, COUNT(*) AS nb
2 FROM journal
3 GROUP BY chemin
4 ORDER BY nb DESC, chemin ASC
5 LIMIT 3;

```

Le second critère de tri (chemin ASC) rend le résultat déterministe en cas d'égalité : sans lui, deux exécutions pourraient renvoyer des lignes différentes.

Question 54.

```

1 SELECT SUBSTR(horodatage, 12, 2) AS heure,
2        COUNT(*) AS total,
3        SUM(CASE WHEN statut BETWEEN 500 AND 599 THEN 1 ELSE 0 END) AS erreurs,
4        ROUND(100.0 * SUM(CASE WHEN statut BETWEEN 500 AND 599 THEN 1 ELSE 0 END)
5              / COUNT(*), 1) AS taux
6 FROM journal
7 GROUP BY heure
8 ORDER BY heure;

```

Deux points méritent attention : la position 12 isole les deux caractères de l'heure dans 'AAAA-MM-JJ HH:MM:SS' (les positions sont comptées à partir de 1) ; et le facteur 100.0 force une division *flottante* — avec 100, la division entière renverrait 0 pour tout taux inférieur à 100 %.

Question 55.

```

1 SELECT ip, COUNT(*) AS nb4xx
2 FROM journal
3 WHERE statut BETWEEN 400 AND 499
4 GROUP BY ip
5 HAVING COUNT(*) >= 3
6 ORDER BY nb4xx DESC;

```

On notera la différence de rôle entre WHERE, qui filtre les *lignes* avant le regroupement, et HAVING, qui filtre les *groupes* après.

Question 56. Volume par pays :

```

1 SELECT c.pays, COUNT(*) AS nb, SUM(j.octets) AS octets
2 FROM journal j
3 JOIN client c ON j.ip = c.ip
4 GROUP BY c.pays
5 ORDER BY octets DESC;

```

Chemins jamais servis avec le statut 200 :

```

1 SELECT DISTINCT chemin
2 FROM journal j1
3 WHERE NOT EXISTS (SELECT 1 FROM journal j2
4                  WHERE j2.chemin = j1.chemin AND j2.statut = 200)
5 ORDER BY chemin;

```

La sous-requête corrélée exprime directement la quantification universelle « pour toute ligne de ce chemin, le statut n'est pas 200 ». Une écriture équivalente et souvent plus efficace utilise l'agrégation :

```

1 SELECT chemin FROM journal
2 GROUP BY chemin
3 HAVING SUM(CASE WHEN statut = 200 THEN 1 ELSE 0 END) = 0;

```

HTTPS

Question 57.

```

1 def puissance_mod(a, e, n):
2     r = 1
3     a = a % n
4     while e > 0:
5         if e % 2 == 1:
6             r = (r * a) % n
7             a = (a * a) % n
8             e = e // 2
9     return r

```

Terminaison. Le variant est e : c'est un entier naturel, et tant que la boucle continue on a $e \geq 1$, donc $\lfloor e/2 \rfloor < e$. Il décroît strictement : la boucle s'arrête. $\square$

Invariant. Notons a_0 et e_0 les valeurs initiales.

Au début de chaque tour, $r \cdot a^e \equiv a_0^{e_0} \pmod{n}$.

Initialisation. $r = 1$, $a \equiv a_0$, $e = e_0$: l'égalité est immédiate.

Hérédité. Notons r', a', e' les valeurs après un tour. Si e est pair, $r' = r$, $a' \equiv a^2$, $e' = e/2$, d'où $r'a'^{e'} \equiv r(a^2)^{e/2} = r a^e$. Si e est impair, $r' \equiv ra$, $a' \equiv a^2$, $e' = (e-1)/2$, d'où $r'a'^{e'} \equiv ra \cdot a^{e-1} = r a^e$. Dans les deux cas la quantité est conservée.

Conclusion. À la sortie $e = 0$, donc $r \equiv r \cdot a^0 \equiv a_0^{e_0}$. $\square$

Nombre de multiplications. Le nombre de tours est le nombre de bits de e , soit $\lfloor \log_2 e \rfloor + 1$. Chaque tour effectue un carré et, au plus, une multiplication : d'où au plus $2(\lfloor \log_2 e \rfloor + 1) = 2\lfloor \log_2 e \rfloor + 2$ multiplications modulaires. Le calcul naïf en demanderait $e - 1$: pour e de 2048 bits, on passe de 2^{2048} à moins de 4100 multiplications.

Vérification : 300 tirages aléatoires avec $n = 2^{61} - 1$ et des exposants jusqu'à 2^{40} redonnent exactement `pow(a, e, n)`.

Question 58. Le client calcule $K_C = B^x \bmod p$ et le serveur $K_S = A^y \bmod p$.

```

1 K_client = puissance_mod(B, x, p)
2 K_serveur = puissance_mod(A, y, p)

```

Démonstration. Par associativité de l'exponentiation dans le groupe $(\mathbb{Z}/p\mathbb{Z})^*$:

$$K_C \equiv B^x \equiv (g^y)^x \equiv g^{xy} \equiv (g^x)^y \equiv A^y \equiv K_S \pmod{p}.$$

$\square$ Le secret g^{xy} n'a jamais transité sur le réseau, bien qu'aucun des deux participants n'ait eu besoin de connaître l'exposant de l'autre.

Application numérique ($p = 23$, $g = 5$, $x = 6$, $y = 15$) :

$$A = 5^6 \bmod 23 = 15625 \bmod 23 = \boxed{8}, \quad B = 5^{15} \bmod 23 = \boxed{19},$$

$$K = B^x \bmod 23 = 19^6 \bmod 23 = \boxed{2} \quad \text{et} \quad A^y \bmod 23 = 8^{15} \bmod 23 = 2.$$

(Détail pour A : $5^2 = 25 \equiv 2$, $5^4 \equiv 4$, $5^6 = 5^4 \cdot 5^2 \equiv 4 \cdot 2 = 8$.)

Question 59. Algorithme naïf. On énumère $k = 0, 1, 2, \dots$ en calculant g^k de proche en proche ($g^{k+1} = g \cdot g^k$) jusqu'à trouver A . Comme l'ordre de g divise $p - 1$, on s'arrête au plus tard après $p - 1$ essais : complexité $O(p)$ en temps, $O(1)$ en espace.

Pas de bébé, pas de géant. Posons $m = \lceil \sqrt{p-1} \rceil$ et écrivons $x = im + j$ avec $0 \leq i, j < m$ (possible car $x < p - 1 \leq m^2$). Alors

$$g^x = A \iff g^j = A(g^{-m})^i.$$

Pas de bébé : on tabule les m valeurs $g^j \bmod p$, $0 \leq j < m$, dans un dictionnaire. *Pas de géant* : on calcule $u = g^{-m} \bmod p$ (par le petit théorème de Fermat, $g^{-m} \equiv g^{p-1-m}$), puis les valeurs Au^i pour $i = 0, 1, \dots$ jusqu'à en trouver une dans la table ; on lit alors j et l'on renvoie $x = im + j$. Coût : $O(\sqrt{p})$ opérations et $O(\sqrt{p})$ en espace.

Ordres de grandeur pour p de 2048 bits. Une machine à 10^9 opérations par seconde effectue environ $3,15 \times 10^{16}$ opérations par an.

naïf : $p \approx 2^{2048} \approx 10^{616}$ opérations, BSGS : $\sqrt{p} \approx 2^{1024} \approx 10^{308}$ opérations.

Même la variante en $\sqrt{p}$ demanderait de l'ordre de 10^{291} années — à comparer aux $1,4 \times 10^{10}$ années de l'âge de l'Univers. Et $O(\sqrt{p})$ en *espace*, soit 10^{308} entrées, est de toute façon physiquement impossible.

Conclusion. Le passage de $O(p)$ à $O(\sqrt{p})$ divise l'exposant par deux : c'est considérable en théorie, sans aucun effet pratique ici. C'est précisément la raison pour laquelle la taille des clés est choisie de sorte que *même* le meilleur algorithme connu — le crible de corps de nombres, en complexité sous-exponentielle — reste hors de portée. C'est aussi pourquoi on préfère aujourd'hui les courbes elliptiques : aucun analogue du crible n'y est connu, si bien qu'une clé de 256 bits y offre le même niveau de sécurité qu'une clé de 3072 bits modulo p , pour un coût de calcul bien moindre.

Question 60. Synthèse.

Confidentialité persistante. Si les exposants x et y sont éphémères et effacés après usage, le secret g^{xy} d'une session passée n'est reconstituable par personne : il ne subsiste nulle part, et la clé privée à long terme du serveur ne sert qu'à *signer* l'échange, jamais à le *chiffrer*. Un attaquant qui aurait enregistré tout le trafic puis obtiendrait la clé privée du serveur — par vol, contrainte légale ou faille — ne pourrait toujours pas déchiffrer les sessions passées ; il pourrait seulement usurper l'identité du serveur pour les sessions *futures*. C'est la *confidentialité persistante* (*forward secrecy*), rendue obligatoire par TLS 1.3.

Dans l'ancien mécanisme, au contraire, le client tirait la clé de session et l'envoyait chiffrée avec la clé publique du serveur : cette clé de session était donc, pour toujours, récupérable par quiconque obtenait la clé privée. Une seule compromission ouvrait rétroactivement des années d'archives — d'où sa suppression pure et simple.

Deux menaces de nature différente. La partie II traite de l'**intégrité** face à un adversaire *aveugle et non malveillant* : le bruit, une mémoire défectueuse, un routeur mal programmé. La somme de contrôle Internet est conçue contre les erreurs *aléatoires*, et l'on a vu qu'elle est très facile à contourner intentionnellement (question 15 : il suffit de permuter deux mots). La partie V traite de la **confidentialité** et de l'**authenticité** face à un adversaire *intelligent*, qui choisit ses altérations pour ne pas être détecté.

Un mécanisme de la première catégorie ne protège en rien contre la seconde menace : c'est pourquoi TLS n'ajoute pas une somme de contrôle mais un *code d'authentification de message* muni d'une clé secrète, dont la sécurité repose sur une hypothèse calculatoire et non sur une propriété combinatoire. Les deux couches coexistent parce qu'elles répondent à des questions distinctes : « ce paquet a-t-il été abîmé ? » et « ce paquet vient-il bien de mon interlocuteur, et lui seul peut-il le lire ? ».